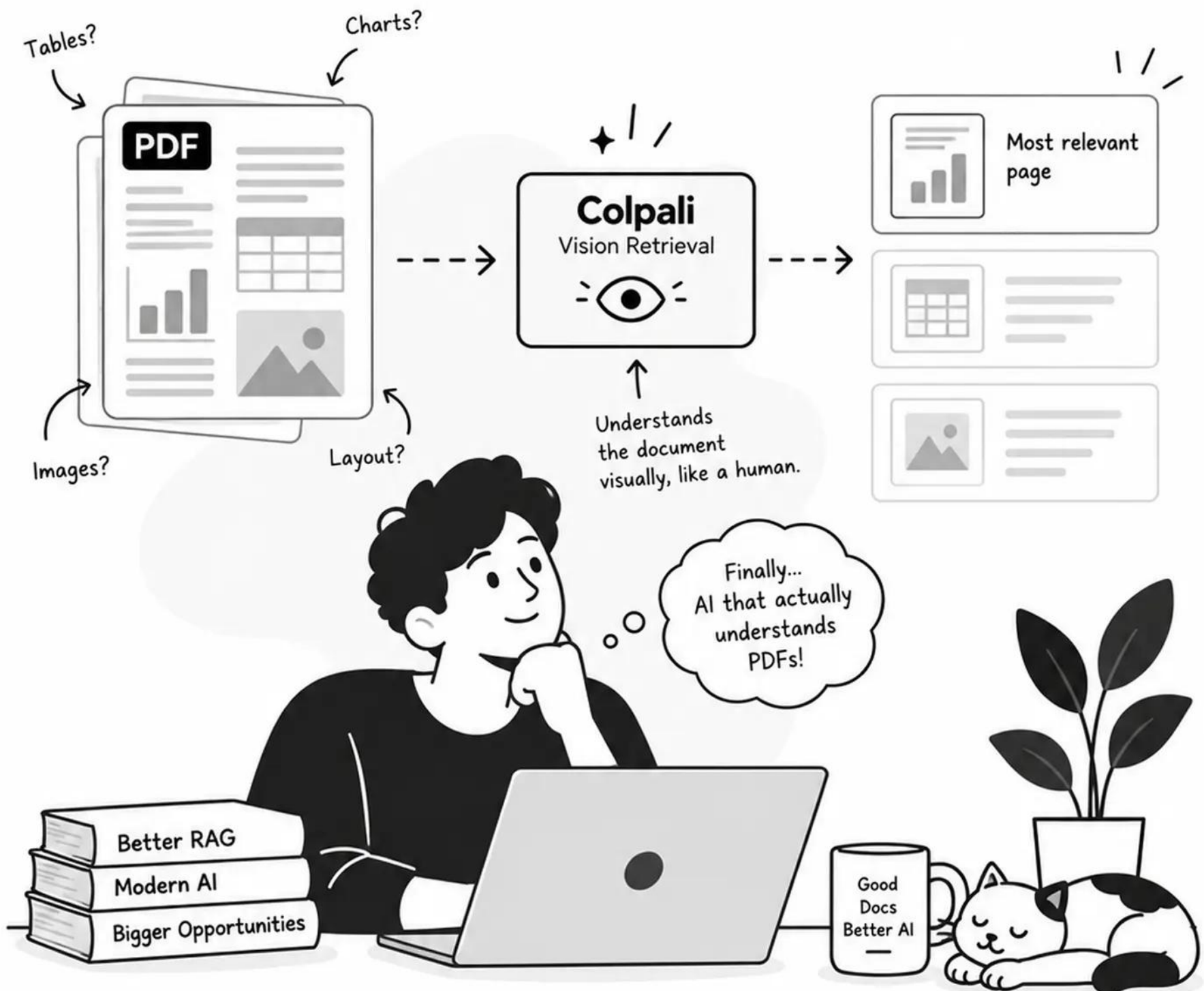


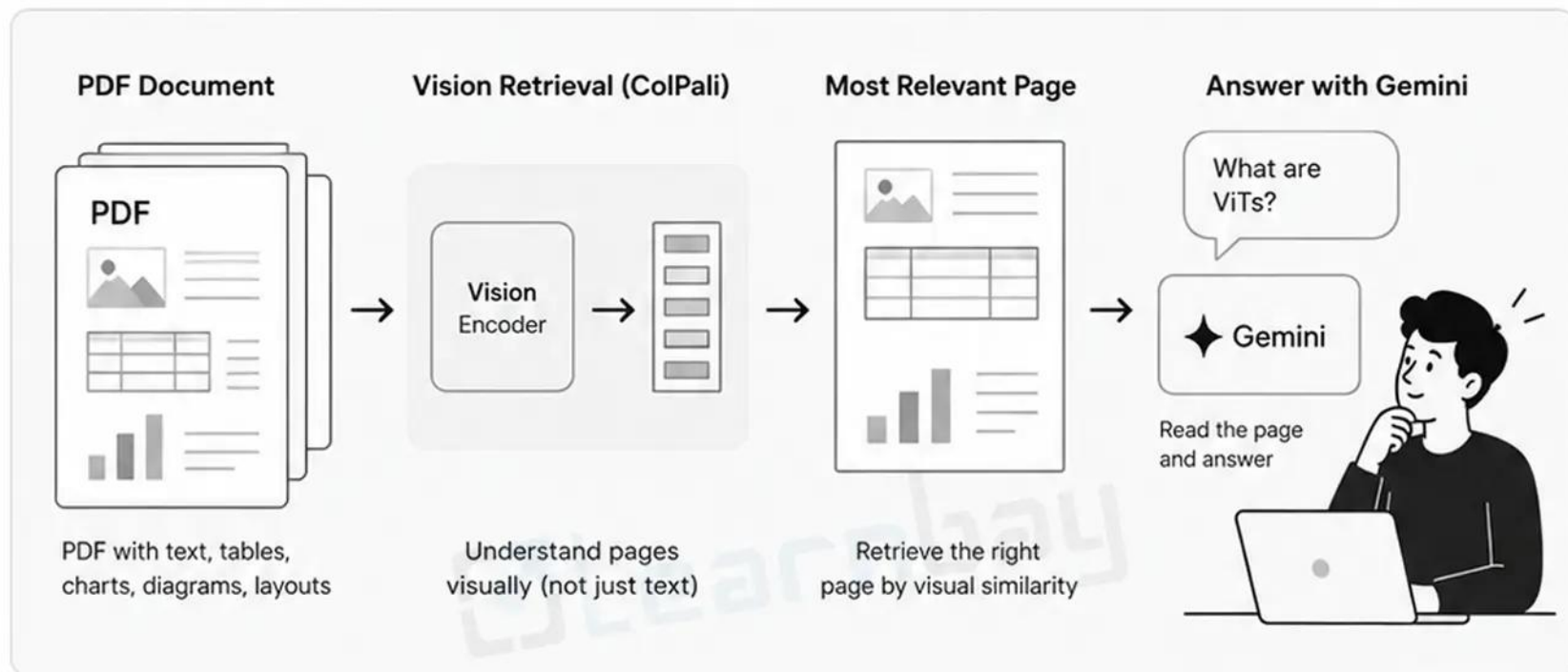
Build a RAG That Reads PDFs Like a Human Colpali vision Retrieval

Your Next AI Project Shouldn't Be Another Chatbot.

Build a RAG that can actually understand complex PDFs.



Build a RAG That Reads PDFs Like a Human - ColPali Vision Retrieval



Stack: `colpali-engine 0.3.15` • `transformers 5.7.0` • `google-genai 1.75.0` • `torch 2.10`

Model: `vidore/colqwen2-v1.0` (Apache 2.0, no token required)

Runtime: Google Colab free tier (T4 16GB) or any GPU with 8GB+ VRAM

What You're Building

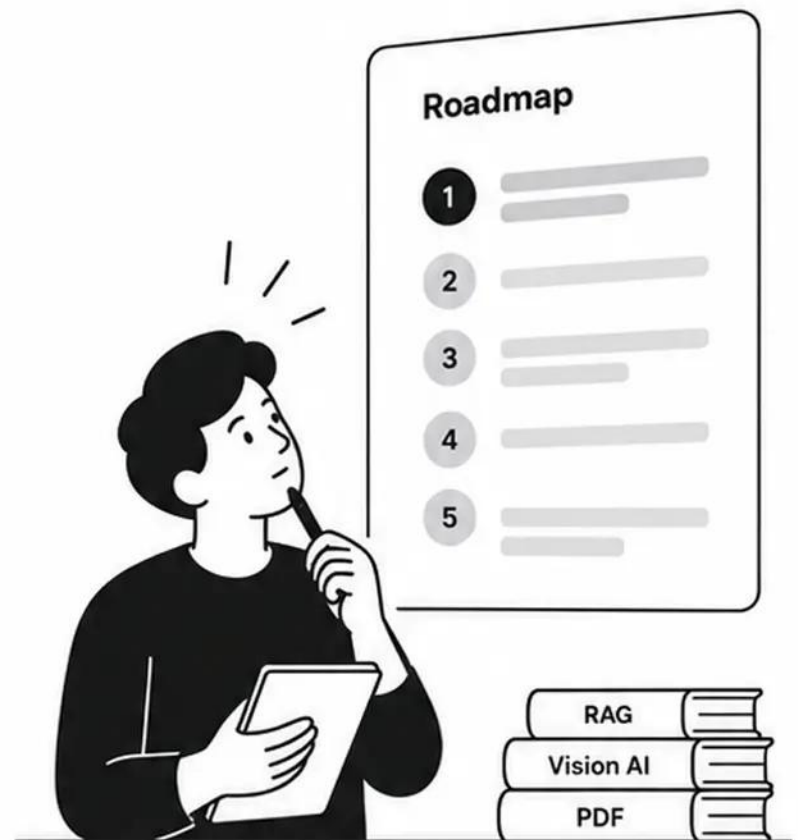
A RAG pipeline that retrieves the right page from a PDF by looking at it - not by reading extracted text. It understands tables, charts, diagrams, and complex layouts that normal RAG misses entirely.

You type a question. The system finds the most visually relevant page. Gemini reads that page and answers you.



Index

1. Why Normal RAG Fails on Real Documents
2. How ColPali Vision Retrieval Works
3. Architecture
4. Project Setup
5. Install Dependencies
6. Convert PDF Pages to Images
7. Load the Vision Retrieval Model
8. Index Your PDF
9. Save and Load the Index
10. Retrieve Relevant Pages
11. Answer with Gemini
12. The Complete Pipeline Script
13. Run It
14. Next Steps
15. Common Errors and Solutions



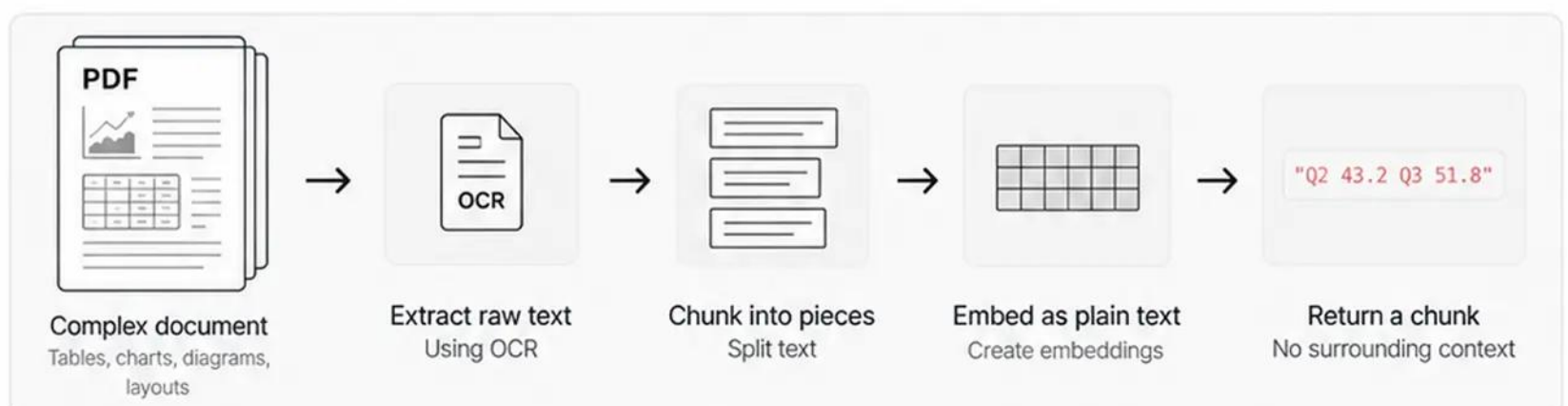
1. Why Normal RAG Fails on Real Documents

Open any annual report, research paper, or legal contract. Now try to answer:
"What was the revenue growth between Q2 and Q3?"

A traditional RAG system will:

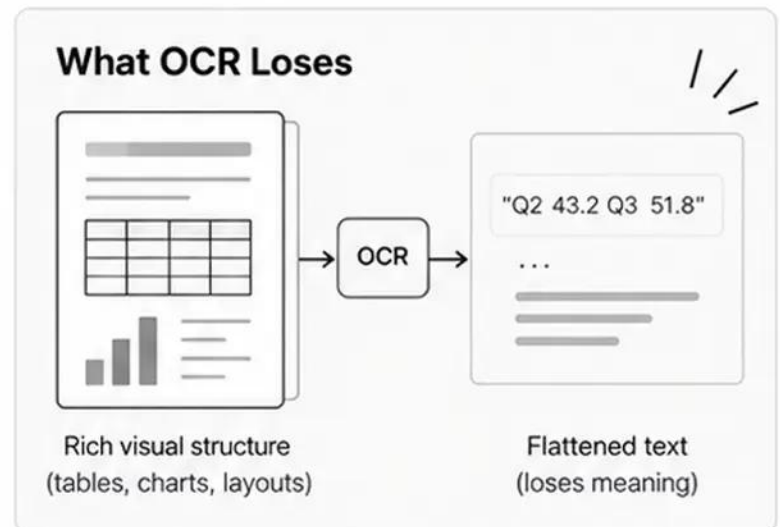
1. Run OCR to extract raw text from the page
2. Chunk that text into pieces
3. Embed those chunks as plain text
4. Return a chunk that might say **"Q2 43.2 Q3 51.8"** with no surrounding context

The table structure is gone. The column headers are gone. The meaning is gone.



This is not a small problem - it's the default failure mode for almost all real business documents. Financial reports, legal filings, scientific papers, and medical records are all built around visual structure that OCR destroys.

The deeper issue is that OCR-based extraction was never designed to understand documents - it was designed to transcribe characters. Every table, chart, sidebar, and diagram it touches loses its spatial meaning in translation.



2. How ColPali Vision Retrieval Works

ColPali (and its successor **ColQwen2**, which we use here) takes a completely different approach. Instead of extracting text, it looks at the entire page as an image and produces embeddings that encode visual meaning.

There are two ideas fused together inside it.

The vision part: ColQwen2 is built on Qwen2-VL, a Vision Language Model. It sees images the way a human does - it understands that a number inside a table cell has a different meaning than the same number in a paragraph.

The retrieval part: ColQwen2 uses ColBERT-style *late interaction*. Rather than collapsing a document into one vector, it produces a separate embedding for every visual patch in the page. When you query, each of your query tokens is matched against every patch, and you take the maximum similarity per query token and sum them up. This is called **MaxSim scoring**.

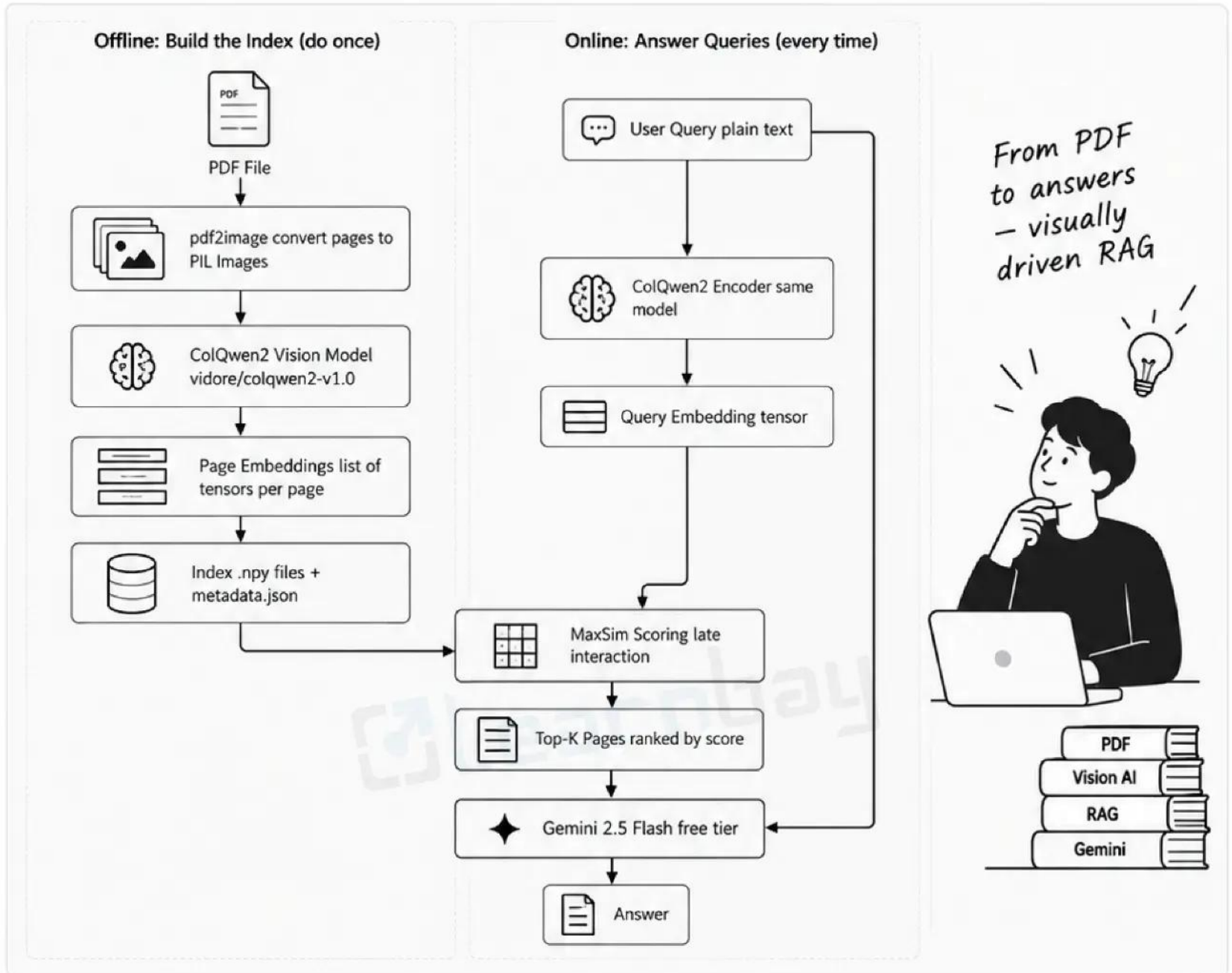
The result: a query like "revenue growth Q2 to Q3" matches the image patches that contain those specific cells in a table - even if no OCR ever touched the page.



What it can handle that normal RAG cannot:

Multi-column tables where row alignment matters	Charts and graphs (line, bar, pie)	Scanned documents with mixed text and figures	Complex layouts: sidebars, footnotes, callout boxes	Handwritten annotations on printed pages	Documents in non-standard fonts

3. Architecture



Offline (do once): PDF → page images → ColQwen2 embeds each page → save index to disk.



Online (every query): query text → ColQwen2 encodes it → MaxSim against stored embeddings → top page sent to Gemini → answer.

4. Project Setup

Create the project and folder structure.



```
uv init vision-rag
cd vision-rag
```

Create the folders your pipeline will use:

```
mkdir pdfs index
```

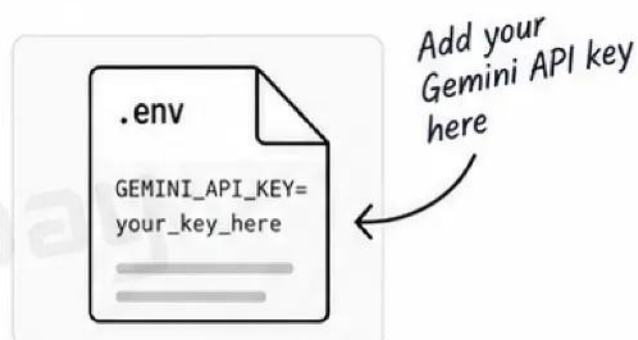
`pdfs/` is where you drop your PDF files. `index/` is where the page embeddings will be saved after ingestion.

Now create the `.env` file for your API key:

```
touch .env
```

Open `.env` and add your Gemini key:

```
GEMINI_API_KEY=your_key_here
```



Get a free Gemini API key at aistudio.google.com. No credit card needed.

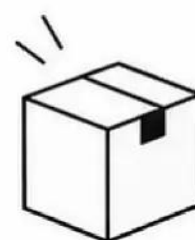
Your final project structure looks like this:

```
vision-rag/
├── .env
├── pdfs/      ← drop your PDF here
├── index/     ← embeddings saved here after ingestion
├── ingest.py  ← converts PDF to index
├── query.py   ← search + answer
└── rag.py     ← shared model loader
```



5. Install Dependencies

```
uv add colpali-engine "transformers==5.3.0" torch torchvision \
```



- ✓ colpali-engine
- ✓ transformers
- ✓ torch
- ✓ torchvision

```
pdf2image Pillow google-genai python-dotenv \
accelerate numpy
```



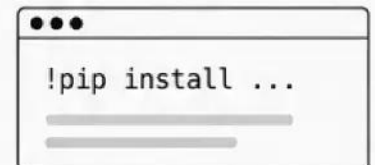
Important: Pin `transformers==5.3.0`. Versions 5.4+ have an internal incompatibility with the `peft` version that `colpali-engine` requires - you'll get an `ImportError: cannot import name '_maybe_shard_state_dict_for_tp' on model load`. 5.3.0 is the last clean version.

If you're on Google Colab, run this instead (Colab uses pip):

```
!pip install colpali-engine "transformers==5.3.0" pdf2image P
illow \
    google-genai python-dotenv accelerate numpy -q
```

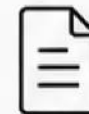


Running on
Google Colab?



Colab also needs `poppler` for the PDF-to-image conversion:

```
!apt-get install -y poppler-utils -q
```



Install poppler
(for PDF to image)

On Windows, poppler is not available via pip or an installer. You need to download the prebuilt binary and put it in your project folder. Do this once:

1. Go to github.com/oschwartz10612/poppler-windows/releases
2. Download the latest `Release-XX.XX.X-X.zip`
3. Extract it - you'll get a folder called something like `Release-25.07.0-0`
4. Rename that folder to `poppler` and place it inside your project root



Your project should look like this:

```
vision-rag/
├── poppler/
│   ├── Library/
│   └── bin/      ← pdftoppm.exe lives here
├── .env
├── pdfs/
├── index/
└── ingest.py
```



Project structure
with poppler

```
└─ query.py
└─ rag.py
```

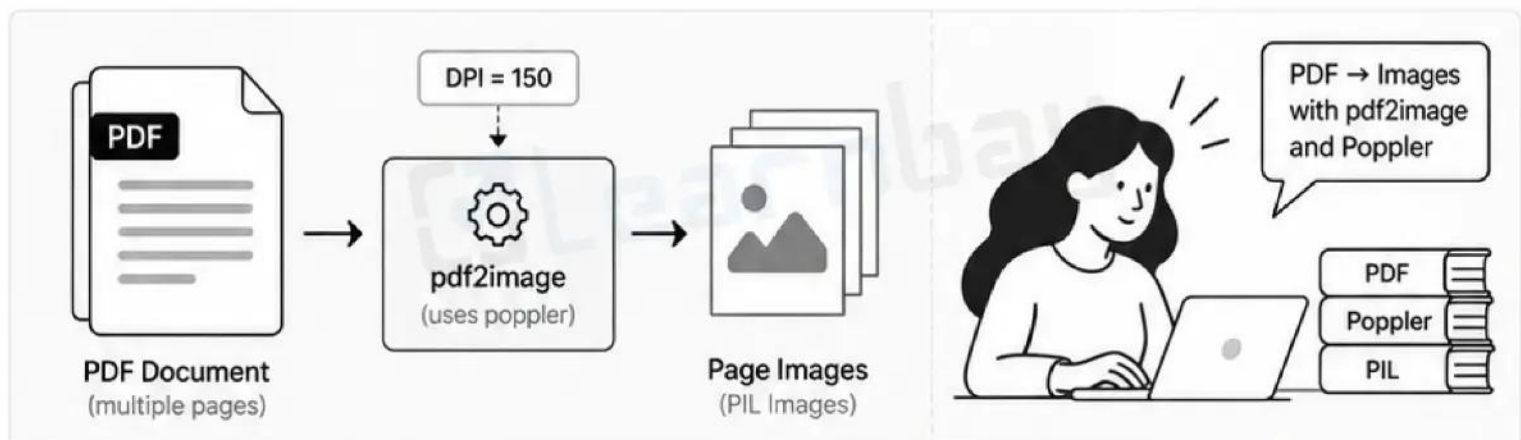
The `pdf_to_images` function in the guide automatically detects this folder and passes the path to `pdf2image`. No changes to your code needed.

Package roles at a glance:

- `colpali-engine` - the ColQwen2 model and processor
- `transformers` - HuggingFace backbone (loaded automatically)
- `pdf2image` - converts PDF pages to PIL Images using poppler
- `google-gemai` - new Google Gemini SDK (v1.75+, replaces google-generativeai)
- `accelerate` - handles device placement (CPU/GPU) automatically

6. Convert PDF Pages to Images

Create `ingest.py`. Start with the imports and the PDF conversion function.



```
# ingest.py
import json
import pathlib
import numpy as np
import torch
from pdf2image import convert_from_path
from PIL import Image
```

The PDF-to-image conversion is a single function call, but there are a few settings worth understanding:

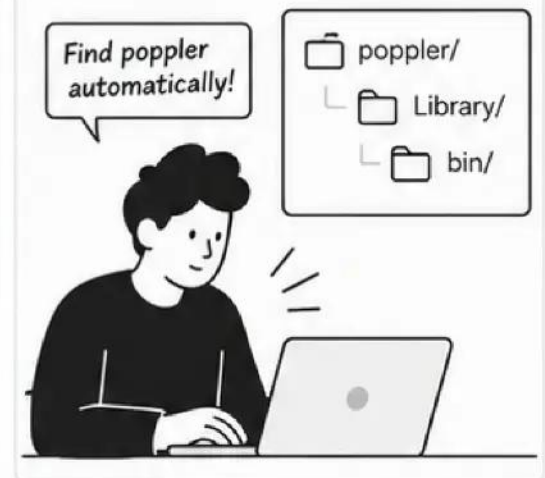
```
def pdf_to_images(pdf_path: str, dpi: int = 150) -> list[Image.Image]:
    # On Windows, point to the extracted poppler bin folder.
    # On Linux/macOS, poppler_path stays None (uses system PATH).
    poppler_path = _get_poppler_path()
```

```

    return convert_from_path(pdf_path, dpi=dpi, fmt="RGB", poppler_path=poppler_path)

def get_poppler_path() -> str | None:
    """Return the path to poppler binaries on Windows, or None on Linux/macOS."""
    local = pathlib.Path("poppler") / "Library" / "bin"
    if local.exists():
        return str(local)
    return None

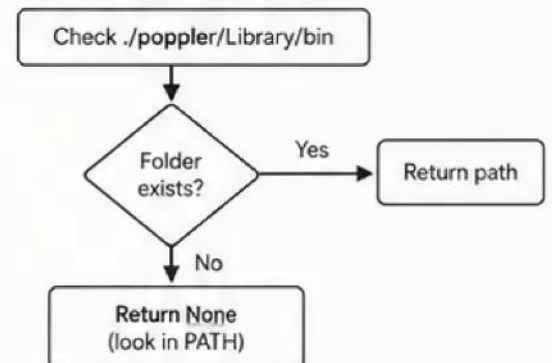
```



Why the `_get_poppler_path` helper?

On Linux and macOS, poppler installs to system PATH and `convert_from_path` finds it automatically. On Windows there is no system-level installer - you download the binaries as a zip and put them in your project folder. The helper checks for that folder first, then falls back to `None` (which means "look in PATH"). This makes the same code work on all platforms without any changes.

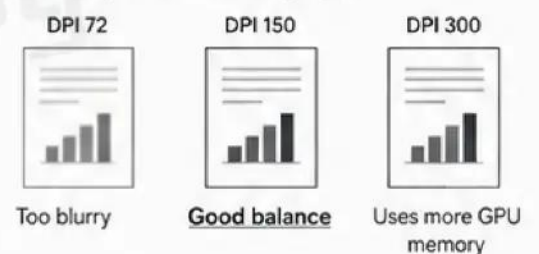
How `_get_poppler_path` works



Why DPI 150?

ColQwen2 accepts images at their natural resolution - it won't resize or warp them. DPI 150 gives you sharp text and clear table lines without creating huge tensors. DPI 72 is too blurry for fine-grained visual retrieval. DPI 300 works but uses significantly more GPU memory.

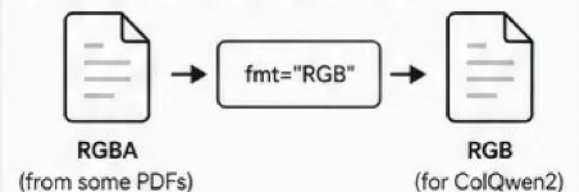
DPI comparison (same page)



Why RGB?

ColQwen2's image processor expects RGB tensors. Some PDFs render as RGBA (transparency channel included). The `fmt="RGB"` argument handles this automatically.

Handle different image formats



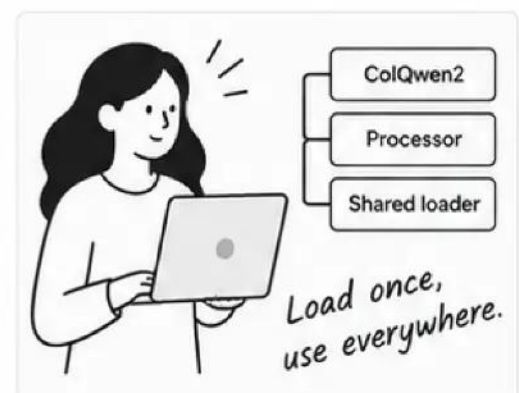
7. Load the Vision Retrieval Model

This is the shared model loader used by both `ingest.py` and `query.py`. Create `rag.py`:

```

# rag.py
import torch
from colpali_engine.models import ColQwen2, ColQwen2Processor

```



The model loader is a single function. The key decisions are device placement and precision:

```
def load_model() -> tuple[ColQwen2, ColQwen2Processor]:
    model_name = "vidore/colqwen2-v1.0"

    device = "cuda" if torch.cuda.is_available() else "cpu"
    dtype = torch.bfloat16 if device == "cuda" else torch.float32

    model = ColQwen2.from_pretrained(
        model_name,
        torch_dtype=dtype,
        device_map=device,
    ).eval()

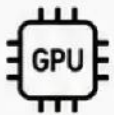
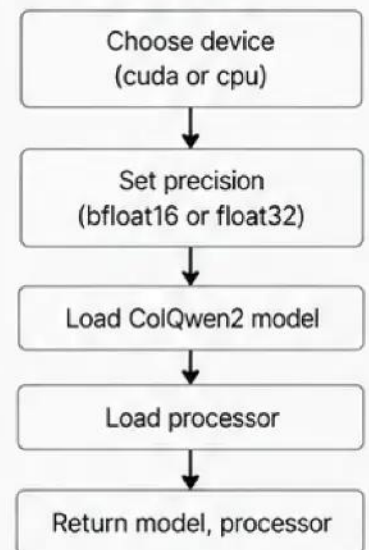
    processor = ColQwen2Processor.from_pretrained(model_name)

    print(f"Model loaded on {device} ({dtype})")
    return model, processor
```

CPU or GPU?
Which precision?
Let's load the model!



Model loading flow



Why bfloat16 on GPU? The model was trained in bfloat16. Running it in float32 on GPU wastes VRAM and gives no benefit. On CPU we use float32 because most CPUs don't support bfloat16 operations natively.



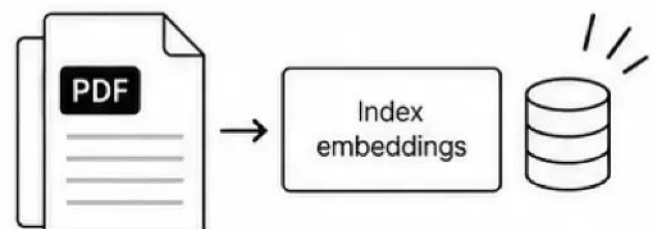
Why `.eval()` ? This disables dropout layers. Without it, you'd get slightly different embeddings every time you run the same input - that makes retrieval non-deterministic.



First run will download the model (~5GB). This is cached in `~/.cache/huggingface/` for all future runs.

8. Index Your PDF

Back in `ingest.py`, add the indexing function below your imports:

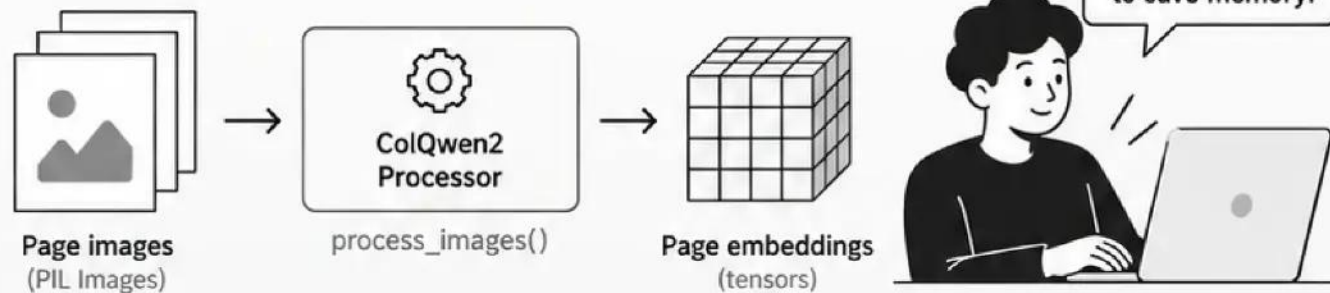


```
from rag import load_model
```

First, the function that embeds a batch of page images:

Embed a batch of page images

Convert page images into visual embeddings using ColQwen2



```
def embed_pages(
    pages: list[Image.Image],
    model: ColQwen2,
    processor: ColQwen2Processor,
    batch_size: int = 2,
) -> list[torch.Tensor]:
    all_embeddings = []

    for i in range(0, len(pages), batch_size):
        batch = pages[i : i + batch_size]

        inputs = processor.process_images(batch).to(model.device)

        with torch.no_grad():
            embeddings = model(**inputs)

        # Move to CPU and detach before storing
        # Each embedding is shape (n_patches, embed_dim)
        for emb in embeddings:
            all_embeddings.append(emb.cpu().detach())

    print(f"Embedded pages {i+1} to {min(i+batch_size, len(pages))}")
    return all_embeddings
```



Why `batch_size=2` ? ColQwen2 uses dynamic resolution - each page can produce a different number of patches. Batch size 2 is safe for a T4 (16GB). Use `batch_size=1` if you get OOM errors.



Why `.cpu().detach()` ?

We store embeddings in RAM, not on the GPU. A 50-page document at 196 patches × 128 dims × float32 is ~5MB - easily held in CPU memory. The GPU is freed for the next batch.

Now the main ingestion function:

```
def build_index(pdf_path: str, index_dir: str = "index") -> None:
    index_path = pathlib.Path(index_dir)
    index_path.mkdir(exist_ok=True)

    print(f"Converting {pdf_path} to images...")
    pages = pdf_to_images(pdf_path)
    print(f"Found {len(pages)} pages")

    print("Loading model...")
    model, processor = load_model()

    print("Embedding pages...")
    embeddings = embed_pages(pages, model, processor)

    # Save each page embedding as a separate .npy file
    pdf_name = pathlib.Path(pdf_path).name
    for idx, emb in enumerate(embeddings):
        np.save(index_path / f"emb_{idx}.npy", emb.numpy())

    # Save metadata: maps embedding index to source page info
    metadata = [
        {"index": idx, "pdf": pdf_name, "page_number": idx + 1}
        for idx in range(len(embeddings))
    ]

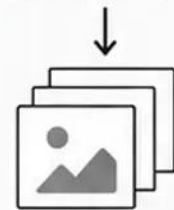
    with open(index_path / "metadata.json", "w") as f:
        json.dump(metadata, f, indent=2)
```

Indexing pipeline

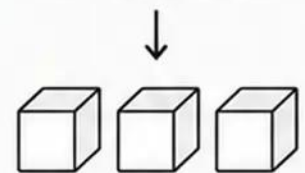
Convert PDF to embeddings and save to disk



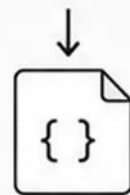
1. Convert PDF to page images



2. Load model and embed pages



3. Save page embeddings (.npy files)



4. Save metadata (metadata.json)



```
print(f"Index saved: {len(embeddings)} pages in '{index_dir}/'")
```

Why separate .npy files instead of one stacked array?

ColQwen2 produces different numbers of patches per page (dynamic resolution). You cannot stack tensors of different shapes into one array. Individual .npy files handle this cleanly.

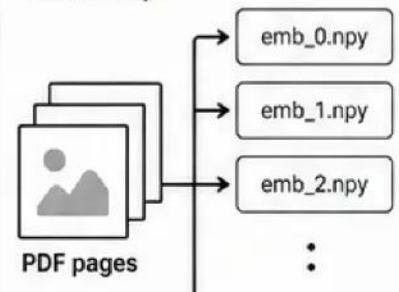
Finally, add the entry point:

```
if __name__ == "__main__":
    import sys
    if len(sys.argv) < 2:
        print("Usage: python ingest.py path/to/document.pdf")
        sys.exit(1)

    build_index(pdf_path=sys.argv[1])
```

Why separate .npy files?

Each page produces a different number of patches, so we save each page embedding individually.



Pages have different shapes (due to dynamic resolution), so we save each embedding as a separate .npy file.



9. Save and Load the Index

The index is just files on disk – you build it once and load it every time you query.

Add the loader to `rag.py` :

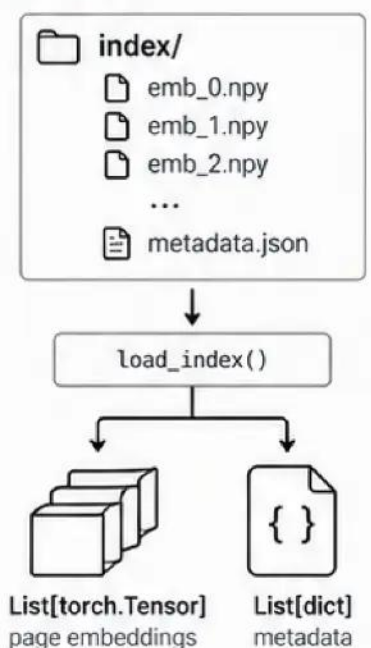
```
# rag.py (continued)
import json
import pathlib
import numpy as np
```

```
def load_index(index_dir: str = "index") -> tuple[list[torch.
Tensor], list[dict]]:
    index_path = pathlib.Path(index_dir)

    if not index_path.exists():
        raise FileNotFoundError(
            f"Index directory '{index_dir}' not found. Run in
            gest.py first."
```

Loading the index

Read .npy files and metadata from disk



```

)

# Load all embeddings in order
np_files = sorted(index_path.glob("emb_*.np"), key=lambda
p: int(p.stem.split("_")[1]))

if not np_files:
    raise FileNotFoundError("No embeddings found. Run ing
est.py first.")

embeddings = [torch.from_numpy(np.load(f)) for f in np_files]

with open(index_path / "metadata.json") as f:
    metadata = json.load(f)

print(f"Loaded index: {len(embeddings)} pages")
return embeddings, metadata

```



The sort key `int(p.stem.split("_")[1])` ensures `emb_10.npy` comes after `emb_9.npy`, not before `emb_2.npy` (which is what lexicographic sort would do).

Loading the index

Read .np files in the correct order and load metadata.



Load .np files
(in numeric order)

Load metadata.json

Embeddings + metadata
(ready for search)



10. Retrieve Relevant Pages

Create `query.py`. Start with imports:

```

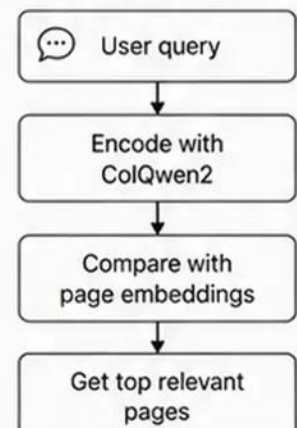
# query.py
import io
import torch
from PIL import Image
from pdf2image import convert_from_path
from rag import load_model, load_index

```

The retrieval function encodes the query text and scores it against every stored page embedding:

Query pipeline (next)

Encode the query and compare against all page embeddings.



```

def retrieve_pages(
    query: str,
    model: ColQwen2,
    processor: ColQwen2Processor,
    embeddings: list[torch.Tensor],
    metadata: list[dict],
    top_k: int = 3,
) -> list[dict]:
    # Encode the query text
    inputs = processor.process_queries([query]).to(model.device)

    with torch.no_grad():
        query_emb = model(**inputs) # shape: (1, n_tokens, embedding_dim)

    # Score against all pages using MaxSim (late interaction)
    scores = processor.score_multi_vector(query_emb, embeddings)

    # scores is shape (1, n_pages) - squeeze the batch dimension
    scores = scores[0].tolist()

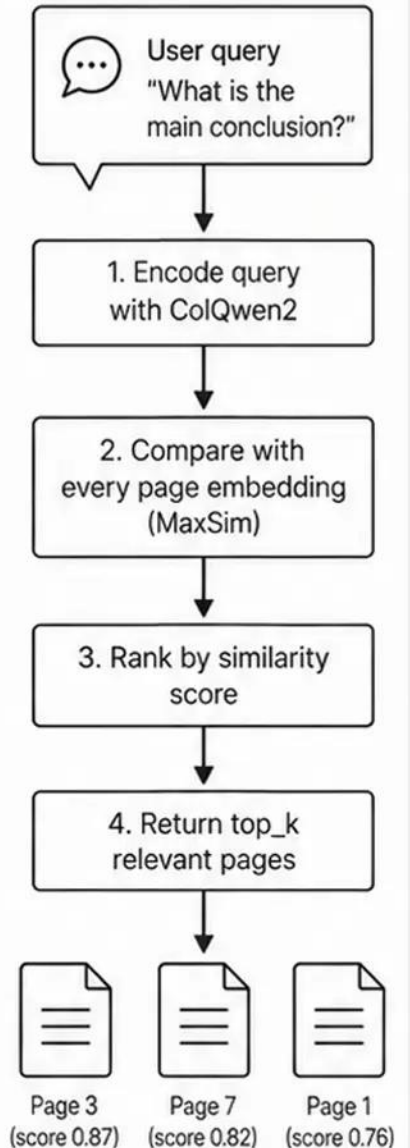
    # Rank pages by score
    ranked = sorted(
        enumerate(scores),
        key=lambda x: x[1],
        reverse=True
    )

    results = []
    for idx, score in ranked[:top_k]:
        results.append({
            **metadata[idx],

```

Query Retrieval Pipeline

Encode the query, compare with all page embeddings, and return the top relevant pages.



15 pages down. The real build starts next.

Want the complete PDF?



Comment **“RAG”** 

and get the full PDF.

Learn
Build
Implement
Your Own RAG



Build a RAG That Reads PDFs Like a Human

ColPali Vision Retrieval



- ✓ Step-by-step guide
- ✓ Complete code
- ✓ Implementation
- ✓ Gemini integration
- ✓ Troubleshooting
- ✓ And more...

